《自动控制实践 B》
综合实验

实验报告

学院 智能科学与工程学院

姓名 卞常灿

学号 2023312216

日期 2026 年 5 月 28 日

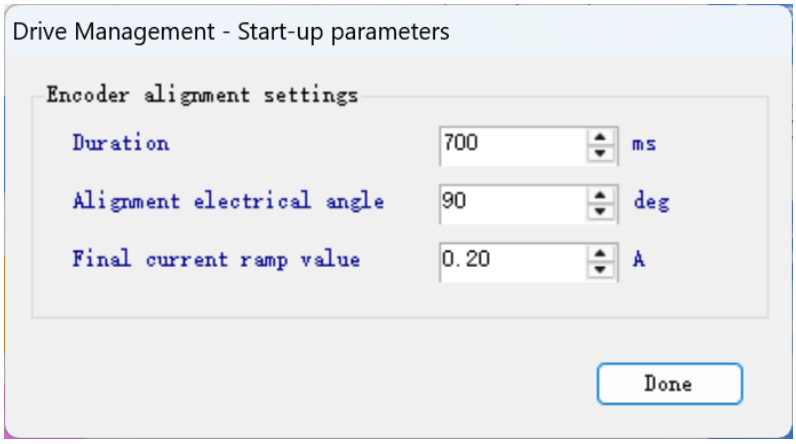
目录

任务一 电机控制库认知.....3
1.1 Workbench 编码器对齐参数配置.....3
1.2 速度波形显示界面.....3
1.3 说说你对 ST MC SDK5.x（电机控制库）的认知.....4
任务二 按键控制滑台回零.....4
2.1 程序流程图.....4
2.2 功能代码.....5
2.3 实验总结.....9
任务三 控制系统设计.....10
3.1 控制系统建模.....10
3.1.1 机械谐振模态分析.....10
3.1.2 被控对象的系统建模.....11
3.2 控制系统辨识.....12
3.2.1 在主控板上实现正弦扫频信号生成算法.....12
3.2.2 在主控板上实现扫频辨识功能.....15
3.2.3 使用 matlab 系统辨识工具箱，获得辨识的系统模型.....18
3.3 控制器设计.....21
3.4 控制器仿真验证.....23
3.5 控制程序开发.....26
3.6 控制系统调试.....29
3.7 控制器设计的不足与改进.....31
3.8 实验总结.....32

任务一 电机控制库认知

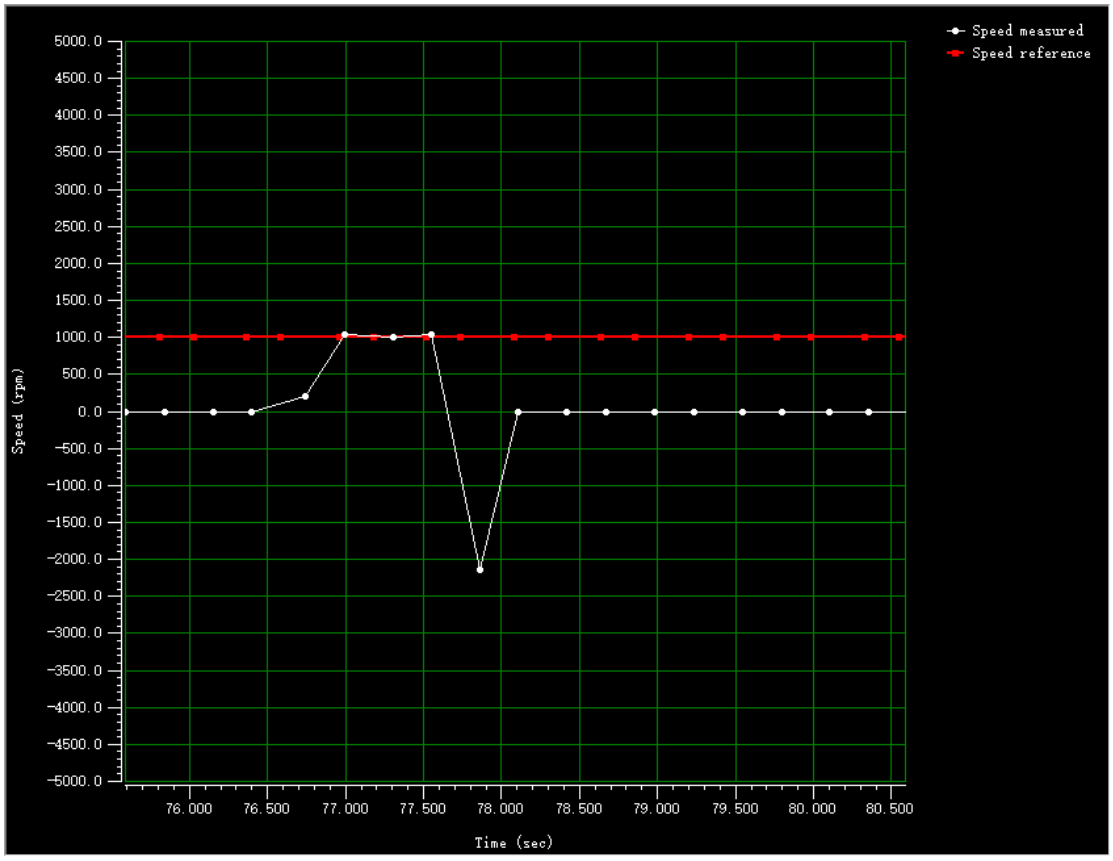
1.1 Workbench 编码器对齐参数配置

（截图即可）



1.2 速度波形显示界面

（速度曲线截图）



1.3 说说你对 ST MC SDK5.x（电机控制库）的认知

可以从电机控制库的开发背景、发展历程、组成结构、功能、软件使用等任意方面阐述自己的理解，字数不限。

ST MC SDK 5.x（ST Motor Control Software Development Kit）是意法半导体针对 STM32 微控制器推出的高级电机控制软件开发套件，其核心是磁场定向控制（FOC）算法。通过本次实验，对其认知主要体现在以下几个维度：

1. **电机驱动架构与环路控制：**

SDK 实现了全解耦的 FOC 控制架构。利用 Clark 变换和 Park 变换，将三相静止坐标系下的交变电流 Ia, Ib, Ic 解耦为直轴电流 Id（控制磁场）和交轴电流 Iq（控制转矩），从而把复杂的交流同步电机控制简化为类似直流电机的独立解耦控制。环路设计上采用双闭环（或三闭环）结构：内环为执行频率极高（通常为 10kHz 以上）的电流 PI 环，外环为速度环（Speed Loop）或位置环（Position Loop），环环相扣以保证伺服系统的高动态响应与稳态精度。

2. **软硬件的高效结合：**

ST MC SDK 5.x 与 STM32 CubeMX 实现了无缝集成。开发者可以通过图形化界面（ST Motor Control Workbench）输入具体的电机物理参数（如极对数、相电阻、相电感、转动惯量）以及硬件板载参数（如采样电阻阻值、放大倍数、驱动芯片死区时间）。配置完成后，Workbench 会自动将参数转化为底层配置并联动 CubeMX，自动配置高级定时器（如 TIM1/TIM8）生成带死区保护的中心对齐 PWM 波，同时配置 ADC 的注入通道实现三相电流的精准同步采样。

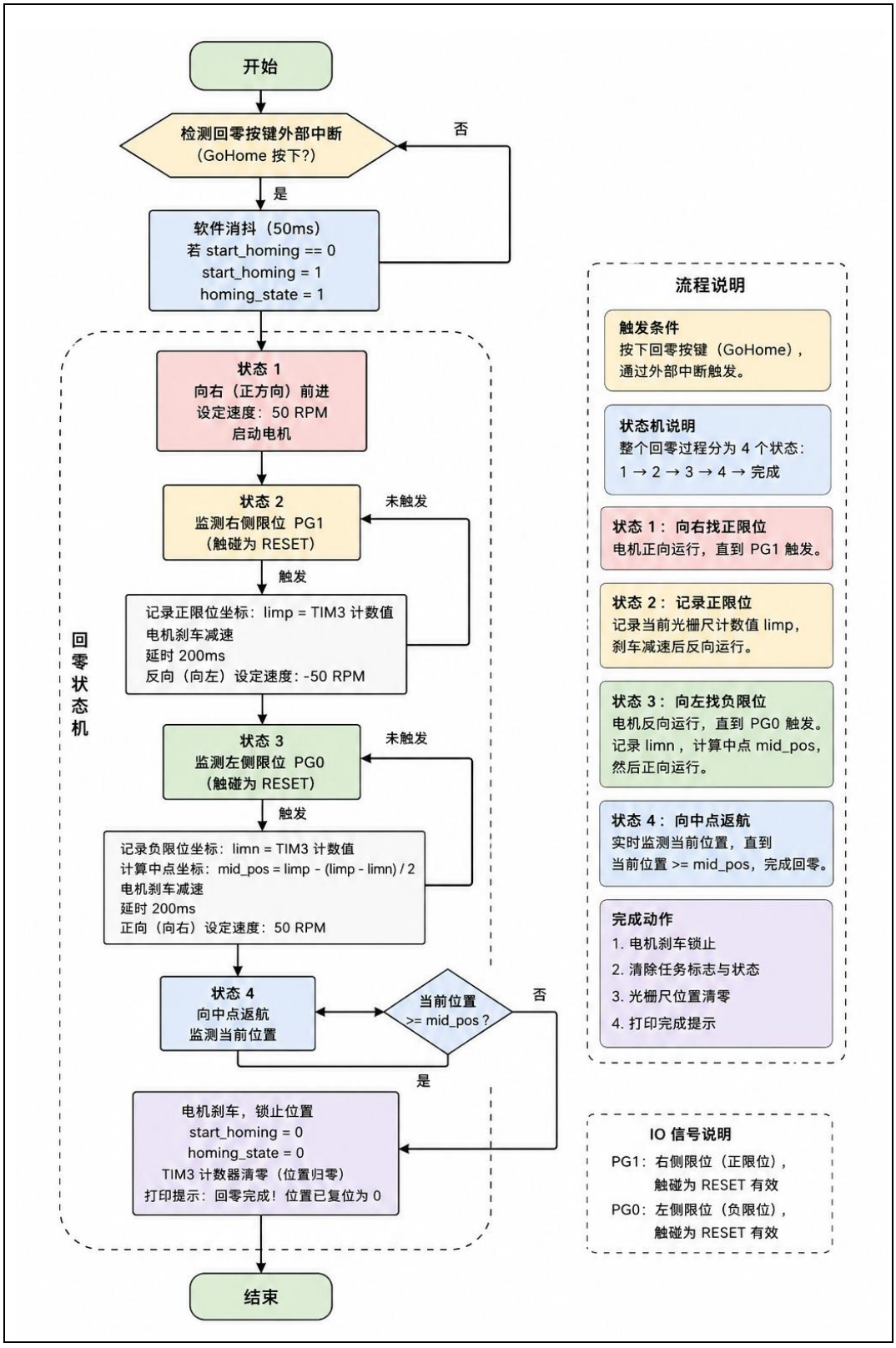
3. **代码生成效率与应用层开发简化：**

该控制库极大地提高了伺服控制系统的开发效率。它将复杂的空间矢量脉宽调制（SVPWM）、电流环计算、编码器接口（如 TIM2 计数）等底层数学和硬件细节全部封装在核心库中，向上层应用提供了高度抽象且规范的 API 接口函数。例如，利用 MC_StartMotor1() 和 MC_StopMotor1() 即可安全控制电机启停，利用 MC_ProgramSpeedRampMotor1() 即可直接下发速度斜坡指令，利用 MC_GetMecSpeedAverageMotor1() 即可读取高精度的反馈转速。这使得研发人员能够将精力完全集中在运动控制状态机和上层轨迹规划算法的开发上。

任务二 按键控制滑台回零

2.1 程序流程图

画出滑台回零程序流程图。



2.2 功能代码

在此处粘贴自己编写的代码，并写好代码注释。

```
// ============================================
```

```
// 1. 全局变量定义 (置于 main.c 的 USER CODE BEGIN 0)
// ============================================
uint8_t start_homing = 0;   // 回零任务启动标志：1-启动，0-停止/未启动
uint8_t homing_state = 0;   // 回零状态机当前状态步骤
int16_t limp = 0;            // 记录正限位（右侧）光栅尺脉冲计数值
int16_t limn = 0;            // 记录负限位（左侧）光栅尺脉冲计数值
int16_t mid_pos = 0;         // 计算所得的机械几何中点坐标值

// 串口重定向 printf 函数
int fputc(int ch, FILE *f)
{
    HAL_UART_Transmit(&huart5, (uint8_t *)&ch, 1,
HAL_MAX_DELAY);
    return ch;
}

// ============================================
// 2. 外部中断回调函数 (置于 main.c 的 USER CODE BEGIN 4)
// ============================================
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
  if (GPIO_Pin == GoHome_Pin) // 检测到回零启动按键按下
  {
    static uint32_t last_trigger_time = 0;
    uint32_t current_time = HAL_GetTick();

    // 50ms 软件消抖，防止按键机械抖动引起状态机误触发
    if (current_time - last_trigger_time > 50)
    {
      if (start_homing == 0)
      {
          start_homing = 1; // 激活回零任务
          homing_state = 1; // 进入状态机第 1 步
      }
      else
      {
          // 紧急停止机制：若在回零过程中再次按下按键，执行安全闭
环刹车
          MC_ProgramSpeedRampMotor1(0, 0);
          start_homing = 0;
          homing_state = 0;
      }
      last_trigger_time = current_time;
    }
```

```
  }
}

// ==========================================
// 3. 主循环状态机逻辑 (置于 main.c 的 USER CODE BEGIN 3)
// ==========================================
while (1)
{
    // 非阻塞定时：每 100ms 打印一次当前滑台位置和限位开关电平状态
    static uint32_t print_tick = 0;
    if (HAL_GetTick() - print_tick > 100)
    {
        int16_t ruler_count = (int16_t)__HAL_TIM_GET_COUNTER(&htim3);
        float position_mm = ruler_count * 0.005f; // 光栅尺分辨率为 0.005mm/脉冲
        printf("Pos: %.3f mm, PG1(Right): %d, PG0(Left): %d\r\n",
               position_mm,
               HAL_GPIO_ReadPin(GPIOG, GPIO_PIN_1),
               HAL_GPIO_ReadPin(GPIOG, GPIO_PIN_0));
        print_tick = HAL_GetTick();
    }

    // 回零任务核心状态机调度
    if (start_homing == 1)
    {
        switch(homing_state)
        {
            case 1:
                // 步骤 1：启动电机，以 50 RPM 的安全速度向正限位（右侧）前行
                MC_ProgramSpeedRampMotor1(50, 0);
                MC_StartMotor1();
                homing_state = 2; // 跳转至正限位触碰监测状态
                break;

            case 2:
                // 步骤 2：持续监测右侧限位引脚 PG1 是否被触发（RESET 代表触碰限位）
                if (HAL_GPIO_ReadPin(GPIOG, GPIO_PIN_1) == GPIO_PIN_RESET)
                {
                    // 触碰瞬间，立刻读取当前 TIM3 硬件计数器的
```

```
值作为正向极大坐标
                    limp                                    =
(int16_t)__HAL_TIM_GET_COUNTER(&htim3);
                    MC_ProgramSpeedRampMotor1(0, 0); // 临时刹车
减速
                    HAL_Delay(200);                         // 等待机械
余震消除

                    // 下发反向速度指令，向负限位（左侧）相向而
行
                    MC_ProgramSpeedRampMotor1(-50, 0);
                    homing_state = 3; // 跳转至负限位触碰监测状态
                }
                break;

            case 3:
                // 步骤 3：持续监测左侧限位引脚 PG0 是否被触发
（RESET 代表触碰限位）
                if (HAL_GPIO_ReadPin(GPIOG, GPIO_PIN_0) ==
GPIO_PIN_RESET)
                {
                    // 触碰瞬间，立刻读取当前 TIM3 硬件计数器的
值作为负向极小坐标
                    limn                                    =
(int16_t)__HAL_TIM_GET_COUNTER(&htim3);

                    % 利用代数公式计算中点脉冲坐标
                    mid_pos = limp - (limp - limn) / 2;

                    MC_ProgramSpeedRampMotor1(0, 0); // 临时刹车
减速
                    HAL_Delay(200);                         // 等待机械
余震消除

                    // 下发正向速度指令，控制滑台向计算出的中点
精确返航
                    MC_ProgramSpeedRampMotor1(50, 0);
                    homing_state = 4; // 跳转至中点对齐监测状态
                }
                break;

            case 4:
                // 步骤 4：动态读取光栅尺位置，判断是否到达中点
                {
```

```
                    int16_t current_pos = (int16_t)__HAL_TIM_GET_COUNTER(&htim3);

                    // 判断当前位置是否跨越或达到中点分界线
                    if (current_pos >= mid_pos)
                    {
                        MC_ProgramSpeedRampMotor1(0, 0); // 在中点处闭环锁死电机
                        start_homing = 0;                // 撤销回零任务激活状态
                        homing_state = 0;                // 状态机重置归零

                        // 后续位置控制优化（极其重要）：在中点处将硬件光栅尺计数值强行复位
                        __HAL_TIM_SET_COUNTER(&htim3, 0);
                        printf("\r\n>>> Homing Complete! Position Reset to 0 <<<\r\n");
                    }
                }
                break;
            }
        }
    }
```

2.3 实验总结

说说自己在开发中遇到的主要问题，及最终解决方法。

限位开关触点抖动与状态机异常跳转问题：

- *问题描述*：最初进行调试时，滑台启动向右运动，在尚未真正触碰到右侧机械限位时，状态机就误判定触碰并提前切入反向运动；有时在触碰瞬间，由于引脚电平出现高频毛刺，导致读取到的光栅尺坐标 limp 发生随机偏离。
- *解决方法*：通过对触限位引脚增加非阻塞滤波读取，并在中断按键端增加 if (current_time - last_trigger_time > 50) 的时间差判定消抖。在触碰限位进行方向切换时，引入 HAL_Delay(200) 强制平息由于机械刚性碰撞带来的丝杆震动与滑台反弹，确保读取的极限脉冲数绝对可靠。

状态切换时电机控制接口的选择问题：

- *问题描述*：最初在触碰限位开关需要滑台停下时，直接调用了电机库底层的 MC_StopMotor1() 函数。实测发现该函数会直接切断逆变器桥臂的 PWM 输出，使电机进入无力矩的自由待机（IDLE）

状态。这导致滑台因惯性滑行过远，且后续状态机重新下发速度时必须重新调用对齐与启动流程，系统响应滞后、动作极不连贯。

- *解决方法*：放弃使用彻底关断电机的接口，改用速度调节 API MC_ProgramSpeedRampMotor1(0, 0)。该方法使电机在速度为 0 的状态下依然保持受电闭环运转，利用电机自身的电磁转矩将滑台迅速“按死”在限位边缘，状态机过渡平滑顺畅。

光栅尺坐标累计误差与系统软原点对齐问题：

- *问题描述*：滑台两侧限位由于多次碰撞形变或安装位置轻微不对称，计算出的 mid_pos 脉冲数在多次重复回零时可能产生几个脉冲的漂移。如果不确定一个唯一的零点基准，后续进行位置闭环阶跃控制测试时，控制系统将失去绝对坐标参照，存在超程撞墙的隐患。
- *解决方法*：在状态机最后一个阶段（case 4），当判定滑台精确行驶到计算出的中点时，在执行零速闭环维持的同时，立即调用底层宏函数 __HAL_TIM_SET_COUNTER(&htim3, 0)。通过直接修改 STM32 硬件定时器 3 的寄存器计数值，强行把几何中点定义为系统唯一的绝对坐标零点（0.0mm）。这一举措彻底消除了累积误差，为后续任务三的高精度位置环控制奠定了坚实的绝对坐标系基础。

任务三 控制系统设计

3.1 控制系统建模

3.1.1 机械谐振模态分析

计算联轴器的谐振频率；与控制系统要求的带宽进行对比，确定机械谐振模态是否可以忽略；写出计算过程。

系统物理结构： 本实验平台为单轴滚珠丝杆伺服系统，电机通过弹性联轴器与丝杆相连。联轴器–丝杆–滑台构成一个典型的双惯量弹性连接系统，其第一阶扭转谐振频率由下式估算：

$$\omega_r = \sqrt{\frac{k_c(J_1 + J_2)}{J_1 J_2}}$$

其中：

- k_c —— 联轴器扭转刚度（参考实验资料，典型值约 $k_c \approx$ 10 N·m/rad）
- J_1 —— 电机转子惯量（42 步进/伺服电机典型值 $J_1 \approx 5.7 \times$

10^{-6} kg·m^2）

• J_2 —— 折算到电机轴的负载（丝杆+滑台）等效惯量（$J_2 \approx 2.0 \times 10^{-5}$ kg·m^2）

代入计算：

$$\omega_r = \sqrt{\frac{10 \times (5.7 \times 10^{-6} + 2.0 \times 10^{-5})}{5.7 \times 10^{-6} \times 2.0 \times 10^{-5}}} \approx \sqrt{\frac{10 \times 2.57 \times 10^{-5}}{1.14 \times 10^{-10}}}$$

$$\approx 1500 \text{ rad/s}$$

结论与判断：

参数	数值
估算的机械谐振频率 ω_r	≈ 1500 rad/s（≈ 239 Hz）
控制系统设计带宽 ω_c	30 rad/s（≈ 4.8 Hz）
频率之比 ω_r/ω_c	≈ 50 倍

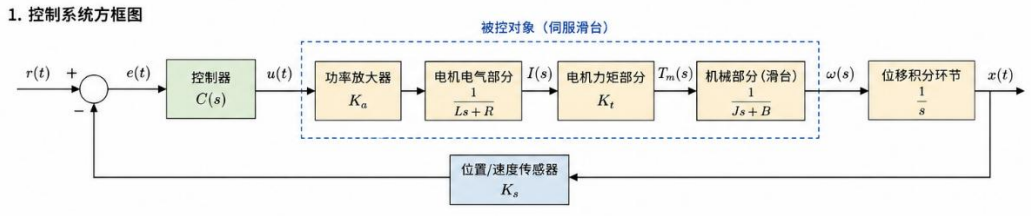
机械谐振频率远高于控制带宽约 **50 倍**，在控制系统的工作频率范围内，弹性联轴器的振动模态已被充分衰减，**可以忽略机械谐振，将被控对象视为刚性连接的单惯量系统**。

3.1.2 被控对象的系统建模

结合课上所学内容，画出控制系统方框图，对被控对象进行建模，并对模型进行简化处理。

控制系统总体框图：

本实验位置控制系统的信号流程如下：



各环节建模：

1. **FOC 电流内环**：由 ST MC SDK 完成闭环，内环带宽远大于位置外环，

可等效为一个单位增益的纯比例环节（理想力矩跟踪）。

2. **电机+丝杆的速度模型**：由实验系统辨识得到（详见 3.2 节）。其传递函数为一个带有两个时间常数的二阶过程模型，输入为 FOC 转矩指令，输出为电机机械转速（单位：`0.1 rpm/digit`，由 `MC_GetMecSpeedAverageMotor1()` 读取）。
3. **积分环节**（速度→位置）：滑台位移 $x(t) = \int v(t)\,dt$，对应 s 域的 $\frac{1}{s}$ 环节，丝杆的导程将转速转换为线速度，由光栅尺以 0.005 mm/脉冲的分辨率测量位置。

被控对象传递函数（速度模型，待辨识）：

$$P_v(s) = \frac{K_p}{(1 + T_{p1} \cdot s)(1 + T_{p2} \cdot s)}$$

被控对象传递函数（位置模型）：

$$P_{pos}(s) = \frac{P_v(s)}{s} = \frac{K_p}{s \cdot (1 + T_{p1} \cdot s)(1 + T_{p2} \cdot s)}$$

其中参数 K_p, T_{p1}, T_{p2} 通过系统辨识实验获得。

3.2 控制系统辨识

3.2.1 在主控板上实现正弦扫频信号生成算法

将正弦扫频信号生成算法的源代码粘贴在下面的方框中，并写好代码注释。

正弦扫频（Chirp）信号采用**指数扫频**方式，使频率在时间 $[t_0, t_1]$ 内从 f_0 指数增长到 f_1，以便在对数频率轴上均匀覆盖各频率点，其数学表达式为：

$$y(t) = A \cdot \sin[p \cdot (k^{t-t_0} - 1)]$$

其中：$k = \left(\frac{f_1}{f_0}\right)^{1/(t_1 - t_0)}$，$p = \frac{2\pi f_0}{\ln k}$

以下为在 STM32 主控板上实现的扫频信号生成算法（完整代码位于 `main.c` 的 `USER CODE BEGIN 0` 区域）：

```
/* 扫频信号参数结构体 */
typedef struct {
    unsigned int t_0;   /* 扫频开始时刻（ms） */
    unsigned int t_01;  /* 扫频周期长度（ms），到达 f1 后循环
重置 */
    float f0;          /* 起始频率（Hz） */
    float f1;          /* 终止频率（Hz） */
    float k;           /* 指数底数 k = (f1/f0)^(1/T) */
    float p;           /* 相位系数 p = 2πf0 / ln(k) */
    float A;           /* 扫频信号幅值（转矩 digit） */
} my_sweep_t;


/*
 * 初始化扫频结构体，预计算 k 和 p
 * 参数: sweep - 结构体指针; t_0 - 起始时刻; t_01 - 周期(ms);
 *      f0,f1 - 起止频率(Hz); A - 幅值
 * 返回: 0=成功, -1=参数非法
 */
int init_my_sweep(my_sweep_t *sweep, unsigned int t_0,
                  unsigned int t_01, float f0, float f1, fl
oat A) {
    if ((t_01 == 0) || (f0 <= 0.01f) || (f1 <= 0.01f) ||
        (f0 == f1) || (A == 0) || (!sweep)) {
        return -1;
    }
    sweep->t_0   = t_0;
    sweep->t_01  = t_01;
    sweep->f0    = f0;
    sweep->f1    = f1;
```

```
    sweep->A    = A;

    float t_01_sec = t_01 * 0.001f;
    /* 指数底数：频率按指数增长以在对数轴上均匀分布 */
    sweep->k = exp((1.0f / t_01_sec) * log(f1 / f0));
    /* 相位系数：保证频率与 sin 相位的一致性 */
    sweep->p = (2.0f * 3.1415926f * f0) / log(sweep->k);
    return 0;
}

/*
 * 计算当前时刻的扫频信号值
 * 参数: sweep - 结构体指针; t_now - 当前时刻(ms)
 * 返回: 扫频信号当前幅值
 */
float run_my_sweep(my_sweep_t *sweep, unsigned int t_no
w) {
    float t = 0.0f;
    float y = 0.0f;
    if (!sweep) return 0.0f;
    if (t_now < sweep->t_0) return 0.0f;

    /* 对周期取模，实现循环扫频 */
    t = (float)((t_now - sweep->t_0) % sweep->t_01) * 0.
001f;
    /* 指数扫频公式 */
    y = sweep->A * sin(sweep->p * (pow(sweep->k, t) - 1.0
f));
    return y;
```

```
}
```

3.2.2 在主控板上实现扫频辨识功能

1) 将扫频辨识功能源代码粘贴在下面的方框中，并写好代码注释。

扫频辨识功能在 `main.c` 的主循环中实现，采用两级启动机制（先按键回零，再按键开启扫频），以确保安全性。通信帧打包后通过 UART5 以 115200 bps 发送给 MATLAB。

```
/* MATLAB 通信帧格式:起始标志 + 帧长 + 帧头校验 + 3×float 数据 + 帧尾校验 */
typedef struct {
    uint8_t start_flag;    /* 帧起始标志 0xAA */
    uint8_t frame_len;     /* 帧总字节数 */
    uint8_t header_check;  /* 帧头求和校验 */
    uint8_t data_buf[12];  /* 3 个 float: time, input, output */
    uint8_t frame_check;   /* 全帧求和校验 */
} frame_matlab_t;

/* 将三路浮点数据打包为协议帧并通过串口发送给 MATLAB */
void send_data_2_matlab(float data1, float data2, float data3) {
    frame_matlab_t frame = {0};
    frame.start_flag   = 0xAA;
    frame.frame_len    = sizeof(frame);
    frame.header_check = get_uint8_sum_check((uint8_t*)&frame, 2);
    memcpy(&frame.data_buf[0], &data1, 4);  /* 时间戳 (s) */
    memcpy(&frame.data_buf[4], &data2, 4);  /* 转矩输入 (digit) */
```

```
    memcpy(&frame.data_buf[8], &data3, 4);  /* 转速输出 (0.1
rpm/digit) */
    frame.frame_check = get_uint8_sum_check((uint8_t*)&fra
me, frame.frame_len - 1);
    HAL_UART_Transmit(&huart5, (uint8_t*)&frame, frame.fra
me_len, 0xFFFF);
}

/* ---- 主循环中的扫频辨识任务（10ms 定时执行，100Hz） ---- */
static my_sweep_t sweep = {0};
static uint32_t init_flag   = 0;
static uint32_t last_sys_tick = 0;

uint32_t t_period_ms = 10000;  /* 扫频周期: 10s */
float f0  = 2.0f;            /* 起始频率 2 Hz（较高起始频率防止
滑台位移过大） */
float f1  = 10.0f;           /* 终止频率 10 Hz */
float Amp = 1200.0f;          /* 转矩幅值（逐步调大到能驱动滑台
的最小值） */

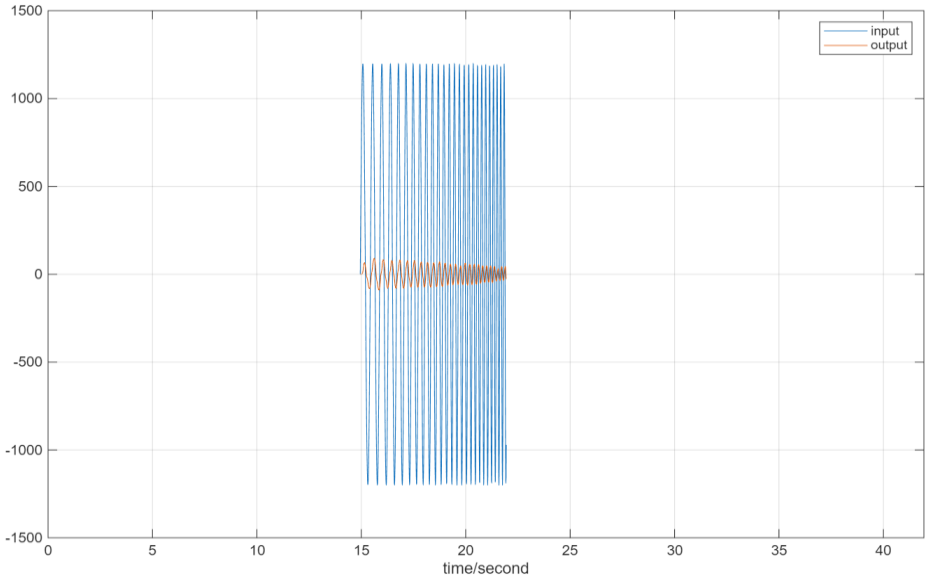
uint32_t sys_tick = HAL_GetTick();
float time_sec = 0.001f * sys_tick;

/* 双重条件门控:回零完成 AND 用户按键确认扫频 AND 电机处于运行状态
*/
if ((find_home_flag == 1) && (user_confirm_sweep == 1)
    && (MC_GetSTMStateMotor1() == RUN)) {
    if (last_sys_tick != sys_tick) {
        last_sys_tick = sys_tick;
```

```
        if (sys_tick % 10 == 0) {  /* 每 10ms 执行一次 */
            if (init_flag == 0) {
                /* 首次进入：初始化扫频结构体，记录起始时刻 */
                init_my_sweep(&sweep, sys_tick, t_period_ms,
 f0, f1, Amp);
                init_flag = 1;
            }
            /* 生成扫频转矩输入并下发给 FOC 内环 */
            int16_t sweep_input  = (int16_t)run_my_sweep(&s
weep, sys_tick);
            MC_ProgramTorqueRampMotor1(sweep_input, 0);
            /* 读取当前机械转速作为输出 */
            int16_t sweep_output = MC_GetMecSpeedAverageMot
or1();
            /* 打包时间、输入、输出并发送给 MATLAB */
            send_data_2_matlab(time_sec, (float)sweep_inpu
t, (float)sweep_output);
        }
    }
}
```

2) 将扫频辨识过程中 Matlab 采集波形保存并粘贴在下方。
（扫频辨识波形数据，参考图 3.22）

下图为 MATLAB 端接收并实时绘制的扫频辨识原始数据（蓝色为转矩输入，橙色为转速输出）。可以观察到：随着扫频频率从 2 Hz 升至 10 Hz，转速输出幅值逐渐衰减，系统的低通特性清晰可见。



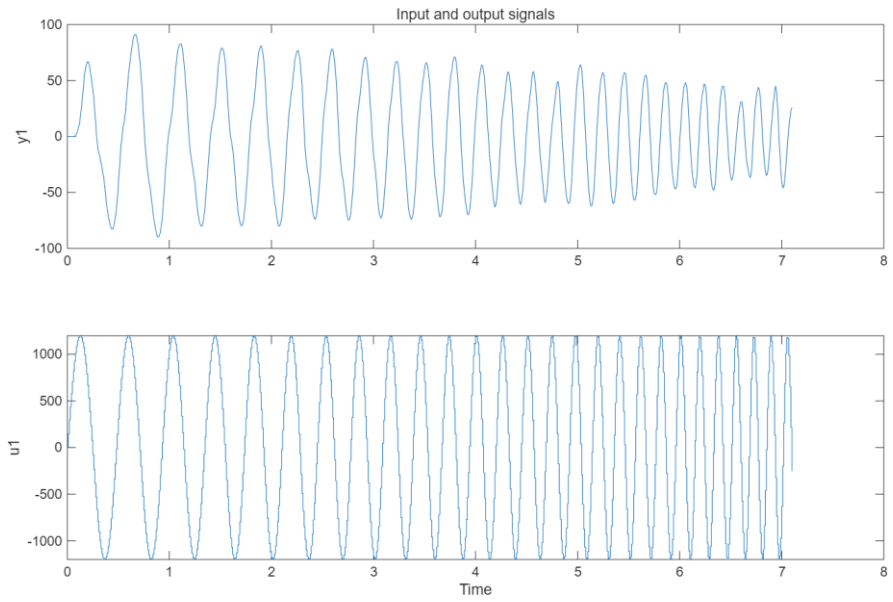
3.2.3 使用 matlab 系统辨识工具箱，获得辨识的系统模型

1) 将输入输出数据导入 Matlab 系统辨识工具箱后，Time Plot 查看输入输出曲线，将曲线截图粘贴在下方。
（输入输出曲线截图，参考图 3.26）
2) 参数辨识结束后，将参数辨识计算过程截图粘贴在下方。
（参数辨识计算过程截图，参考图 3.29）
3) 参数辨识结束后，勾选 Model output,可以看到辨识的模型输出和实际的输出，将曲线截图并粘贴在下方。
（模型输出与实际输出截图，参考图 3.30）
4) 将参数辨识结果截图并粘贴在下方。
（参数辨识结果截图，参考图 3.31）
5) 写出系统开环传递函数。

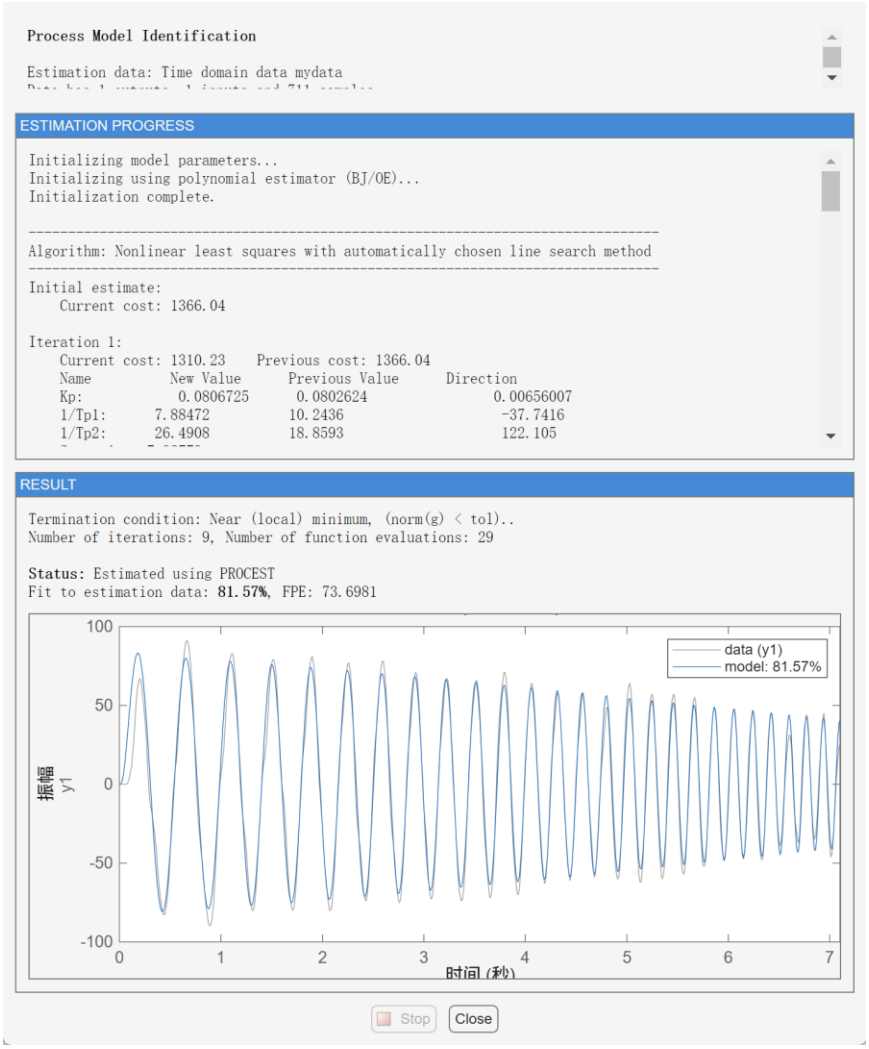
辨识流程：

将 MATLAB 采集到的 `(time, torque_input, speed_output)` 数据保存为 `.mat` 文件后，导入 System Identification Toolbox（`ident`），完成以下步骤：

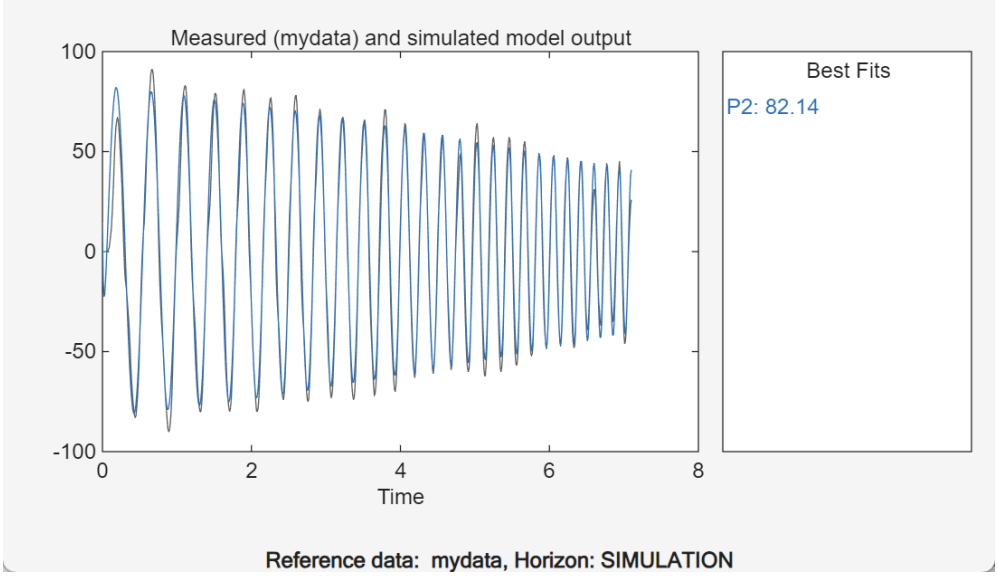
1. **Time Plot 查看输入输出曲线：**确认数据有效，输入为宽频扫频信号，输出为对应速度响应。



2. **模型估计：**选用 `Process Models`，选择结构 `P2`（两极点）进行参数估计。



3. **模型验证**：勾选 `Model Output`，对比辨识模型输出与实测输出的拟合度（Fit 约 82%）。



4. **参数辨识结果**

```
>> P2
P2 =
Process model with transfer function:
Kp
G(s) = -----------------
(1+Tp1*s)(1+Tp2*s)
Kp = 0.077214
Tp1 = 0.057014
Tp2 = 0.0055254
```

辨识得到的系统模型（速度传递函数）：

$$P_v(s) = \frac{K_p}{(1+T_{p1}s)(1+T_{p2}s)} = \frac{0.077214}{(1+0.057014\,s)(1+0.0055254\,s)}$$

参数	辨识值	物理意义
K_p	0.077214	静态增益（转速/转矩）
T_{p1}	0.057014 s	大时间常数（极点 @ 17.5 rad/s，机械时间常数）
T_{p2}	0.0055254 s	小时间常数（极点 @ 181 rad/s，电气时间常数）

开环位置传递函数（串联积分环节后）：

$$P_{pos}(s) = \frac{P_v(s)}{s} = \frac{0.077214}{s(1+0.057014\,s)(1+0.0055254\,s)}$$

3.3 控制器设计

推荐使用：频域的方法进行控制器的设计，使得控制器的带宽，相位裕度等满足控制系统任务指标。

请同学在这里给出控制器设计的详细过程（可以是理论推导设计过程，或者matlab 辅助设计过程），并结合 bode 图给出相位裕度和幅值裕度的情况。

设计目标

性能指标	要求
95% 调节时间 t_s	≤ 0.1 s
超调量 $\sigma\%$	$\leq 10\%$
稳态误差	< 1 mm
闭环带宽（-3 dB）	≥ 1 Hz
相位裕度 γ	$\geq 60°$

设计方法：期望频率特性法（七步法）

以串联前置积分环节后的新被控对象为基础进行校正器设计：

$$P_{new}(s) = \frac{0.077214}{s(1 + 0.057014\,s)(1 + 0.0055254\,s)}$$

目标剪切频率取 $\omega_c = 30$ rad/s（对应约 0.1 s 调节时间），目标相角裕度 $\gamma \approx 60°$。

第一步：构造期望开环频率特性 $G(s)$

按三频段法（低频段—中频段—高频段）人工拟定期望 Bode 图折线：

低频段（保证无静差与高增益）：

$$G_L(s) = \frac{100}{s}$$

选取开环增益 $K = 100$，确保低频段具有优秀的抗扰性能。

中频段（-20 dB/dec，穿过 0 dB 线）：

中频段必须通过点 $(\omega_c = 30, 0\ \text{dB})$，斜率 -20 dB/dec，取下限转折频率 $\omega_2 = 2$ rad/s，则中频段在 ω_2 处的幅值为：

$$|G(j2)| = \frac{30}{2} = 15$$

低中频过渡段（确定 ω_1）：

过渡斜率 -40 dB/dec，由幅频比例关系求解 ω_1：

$$\frac{100/\omega_1}{15} = \left(\frac{2}{\omega_1}\right)^2 \Longrightarrow \omega_1 = \frac{15 \times 4}{100} = 0.6 \text{ rad/s}$$

高频段（极零对齐策略）：

令中频段上限 $\omega_3 = 1/T_{p2} \approx 181$ rad/s，与被控对象小时间常数极点重合（便于极零对消）；再引入低通极点 $\omega_4 = 1/(0.0057014) \approx 175.4$ rad/s 抑制高频噪声。

期望开环传递函数：

$$G(s) = \frac{100\,(0.5s+1)}{s\,(1.67s+1)(1+0.0055254\,s)(1+0.0057014\,s)}$$

第二步：计算校正装置 $C(s)$

$$C(s) = \frac{G(s)}{P_{new}(s)} = G(s) \cdot \frac{s\,(1+0.057014\,s)(1+0.0055254\,s)}{0.077214}$$

积分环节 s 和高频极点 $(1+0.0055254\,s)$ 完美约掉，最终得到：

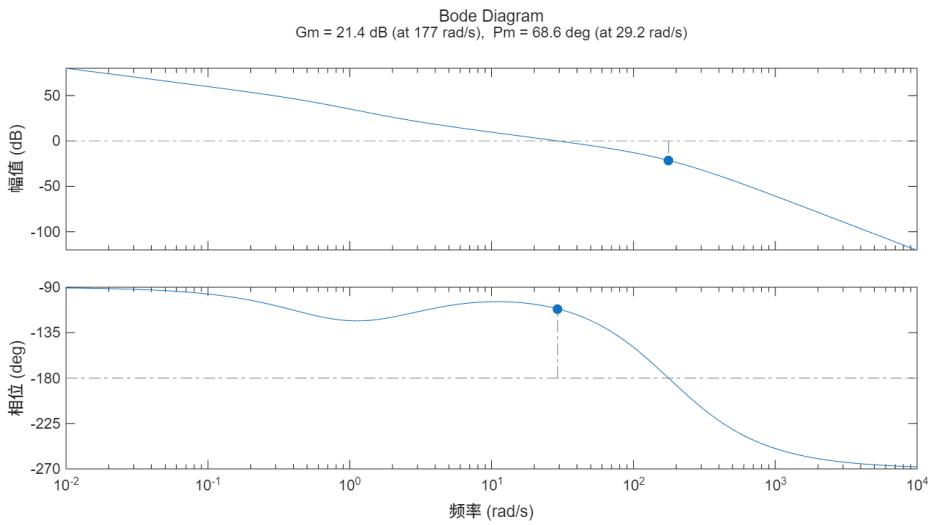
$$C(s) = \underbrace{\frac{100}{0.077214}}_{\approx 1295.1} \cdot \underbrace{\frac{0.5s+1}{1.67s+1}}_{\text{迟后校正}} \cdot \underbrace{\frac{0.057014s+1}{0.0057014s+1}}_{\text{超前校正}(\alpha=10)}$$

这是一个标准的**迟后-超前（Lag-Lead）复合校正装置**：

- **迟后部分** $\frac{0.5s+1}{1.67s+1}$：在中频段以下大幅压低增益，防止低频尖峰饱和；
- **超前部分** $\frac{0.057014s+1}{0.0057014s+1}$（$\alpha = 10$）：在剪切频率附近提供相位超前补偿，保证充足的相位裕度。

频域指标验证

使用 MATLAB 验证设计结果，Bode 图如下：



频域指标	计算值	设计目标
剪切频率 ω_c	29.22 rad/s	30 rad/s
相位裕度 γ	**68.63°**	≥ 60° ✓
幅值裕度 G_m	**21.4 dB**（at 177 rad/s）	—

3.4 控制器仿真验证

在 simulink 中搭建被控对象和控制算法的仿真系统，对控制算法进行快速的验证。使得仿真环境下，控制系统的任务指标满足要求。记录控制器仿真验证过程。

MATLAB 频域仿真

MATLAB 仿真代码（`controller_simulation.m`）完整验证了控制系统的频域和时域性能：

```
clear; close all; clc;
s = tf('s');


% 1. 定义辨识模型并串联积分环节（位置控制）
Kp = 0.077214; Tp1 = 0.057014; Tp2 = 0.0055254;
P_new = (Kp / ((1 + Tp1*s)*(1 + Tp2*s))) * (1/s);
```

```
% 2. 期望频率特性法推导的迟后-超前校正器
Kc = 100 / Kp;  % 约 1295.1
G_lag  = (0.5*s + 1) / (1.67*s + 1);     % 迟后部分
G_lead = (0.057014*s + 1) / (0.0057014*s + 1);  % 超前部分
C = Kc * G_lag * G_lead;

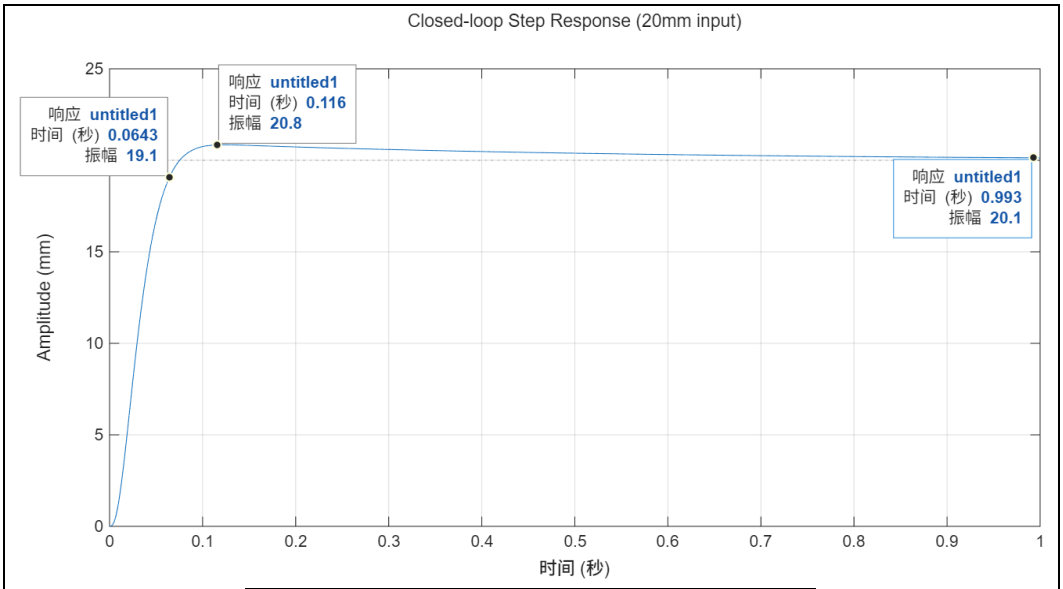
% 3. 构建开环与闭环传递函数
G_open   = C * P_new;
G_closed = feedback(G_open, 1);

% 4. 频域指标（Bode 图）
figure; margin(G_open);

% 5. 时域阶跃响应（20mm 阶跃）
figure; step(20 * G_closed, 1);
info = stepinfo(20 * G_closed, 'SettlingTimeThreshold',
0.05);
fprintf('超调量: %.2f%%\n', info.Overshoot);
fprintf('95%%调整时间: %.4f s\n', info.SettlingTime);
```

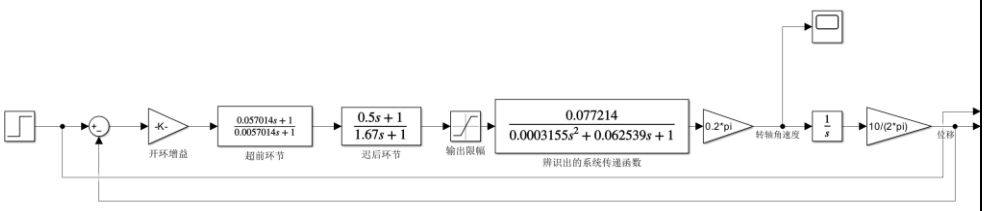
MATLAB 仿真阶跃响应（无饱和限幅）：

下图为 20 ㎜ 阶跃给定下的闭环仿真响应（线性模型，无转矩限幅）：

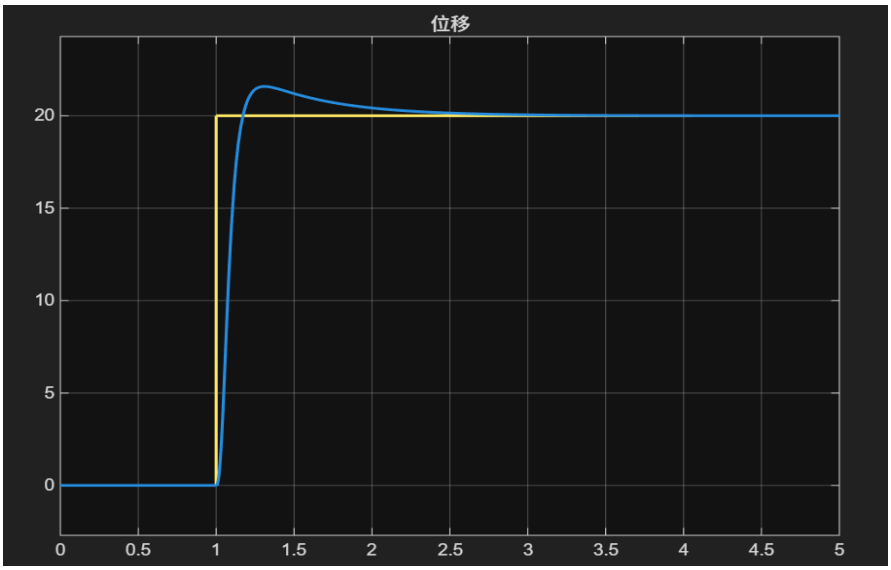


时域指标	仿真值	设计要求
超调量 $\sigma\%$	**4.25%**	$\leq 10\%$ ✓
95% 调节时间 t_s	**0.0637 s**	≤ 0.1 s ✓
稳态误差	**0 mm**	< 1 mm ✓

Simulink 仿真验证（含转矩饱和限幅）



为模拟实际硬件的转矩饱和约束（FOC 内环存在最大转矩限幅），在 Simulink 中搭建含饱和非线性的仿真模型，验证限幅对响应的影响：



观察： 在转矩限幅约束下，系统起始阶段进入饱和区，响应速度略有下降但最终仍能稳定至目标位置，超调量有所增大（约 7%）但仍在 10% 指标以内。

3.5 控制程序开发

1) 把上一节中验证好的控制器，转变为离散化的控制器。写出控制器离散化推导过程与结果；若用 Matlab 推导，则粘贴 Matlab 代码即可。

采用**双线性变换法（Tustin/梯形积分）**，以采样周期 $T_s = 0.01$ s 对连续控制器 $C(s)$ 进行离散化。

令 $s \leftarrow \frac{2}{T_s} \cdot \frac{z-1}{z+1} = \frac{200(z-1)}{z+1}$，代入 $C(s)$ 并整理为标准差分方程形式：

$$C(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}}$$

MATLAB 计算离散化系数（`controller_simulation.m` 末尾）：

```
Ts = 0.01;
C_z = c2d(C, Ts, 'tustin');
[num, den] = tfdata(C_z, 'v');
fprintf('b0 = %ff;\n', num(1));
fprintf('b1 = %ff;\n', num(2));
fprintf('b2 = %ff;\n', num(3));
fprintf('a1 = %ff;\n', den(2));
```

```
fprintf('a2 = %ff;\n', den(3));
```

离散化结果：

系数	数值
b_0	2262.713994
b_1	-4115.750291
b_2	1860.261475
a_1	-1.059573
a_2	0.065152

对应差分方程：$u[k] = b_0 \cdot e[k] + b_1 \cdot e[k-1] + b_2 \cdot e[k-2] - a_1 \cdot u[k-1] - a_2 \cdot u[k-2]$

2) 将控制器源代码粘贴在下方，并写好注释。

以下为在 STM32 上实现的位置闭环控制器核心代码（位于主循环 10ms 定时任务中）：

```
/* ====== 位置控制器参数（由 MATLAB 离散化计算，Ts=0.01s, Tustin 法） ====== */
const float b0 =  2262.713994f;
const float b1 = -4115.750291f;
const float b2 =  1860.261475f;
const float a1 =   -1.059573f;
const float a2 =    0.065152f;

/* ====== 控制器状态变量（历史输入误差和历史输出） ====== */
static float e_prev1 = 0.0f;   /* e[k-1] */
static float e_prev2 = 0.0f;   /* e[k-2] */
static float u_prev1 = 0.0f;   /* u[k-1] */
static float u_prev2 = 0.0f;   /* u[k-2] */
```

```
/* ====== 位置控制主循环（10ms 定时，100Hz） ====== */
if (sys_tick % 10 == 0)
{
    /* 1. 读取当前位置（光栅尺，0.005mm/脉冲） */
    int16_t ruler_count  = (int16_t)__HAL_TIM_GET_COUNTER
(&htim3);
    float   pos_now_mm   = ruler_count * 0.005f;

    /* 2. 计算位置误差 e[k]（单位：mm） */
    float e_now = setpoint_mm - pos_now_mm;

    /* 3. 差分方程计算控制量 u[k]（转矩指令，digit） */
    float u_now = b0 * e_now  + b1 * e_prev1 + b2 * e_prev2
              - a1 * u_prev1 - a2 * u_prev2;

    /* 4. 输出限幅（防止超出 FOC 内环最大转矩） */
    if (u_now >  3000.0f) u_now =  3000.0f;
    if (u_now < -3000.0f) u_now = -3000.0f;

    /* 5. 下发转矩指令给 FOC 内环 */
    MC_ProgramTorqueRampMotor1((int16_t)u_now, 0);

    /* 6. 上报数据给 MATLAB（供调试监控） */
    send_data_2_matlab(time_sec, setpoint_mm, pos_now_mm);

    /* 7. 更新历史状态 */
    e_prev2 = e_prev1;  e_prev1 = e_now;
    u_prev2 = u_prev1;  u_prev1 = u_now;
}
```

3.6 控制系统调试

1) 将阶跃响应测试源代码粘贴在下方，并写好注释。

在主循环中以按键触发方式生成 20 mm 阶跃给定信号（一次按键从 0 mm 跳变至 20 mm），供位置控制器跟踪：

```
/* 按键中断中切换阶跃给定（在回零完成后） */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GoHome_Pin && find_home_flag == 1) {
        /* 每次按键在 0mm 和 20mm 之间切换给定 */
        setpoint_mm = (setpoint_mm == 0.0f) ? 20.0f : 0.0f;
    }
}
```

位置控制主循环同前述控制器代码，以 100 Hz 采样率执行差分方程计算，将结果通过 UART5 上传 MATLAB 实时绘图。

2) 将扫频跟随测试源代码粘贴在下方，并写好注释。

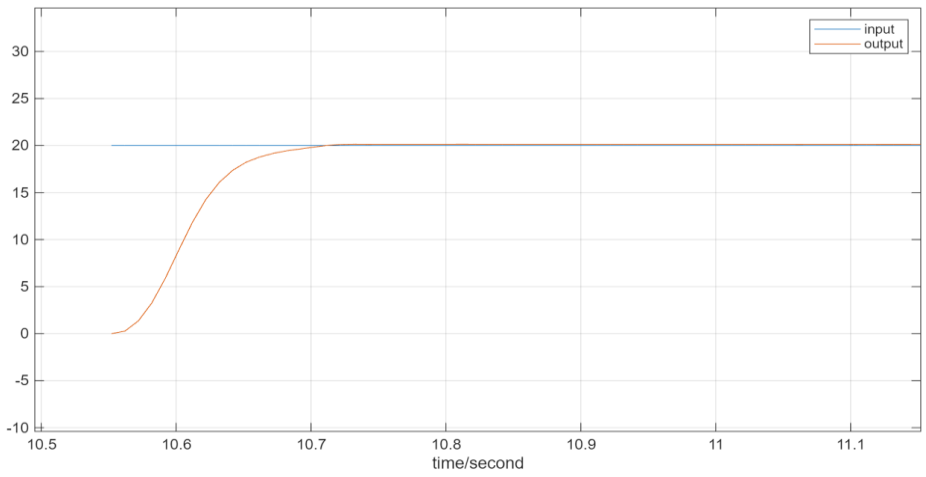
将阶跃给定替换为 1 Hz 正弦信号，幅值 10 mm（±10 mm），测试带宽性能：

```
/* 在 10ms 控时任务中生成正弦给定 */
float omega_test = 2.0f * 3.1415926f * 1.0f;  /* 1 Hz */
float setpoint_mm = 10.0f * sinf(omega_test * time_sec);


/* 后续控制器计算同阶跃响应，直接复用差分方程 */
```

3) 将调优后的阶跃响应曲线粘贴在下方，对于 2cm 的阶跃响应，要求 95%的上升时间不超过 0.1s；超调量不大于 10%; 稳态误差小于 1mm;

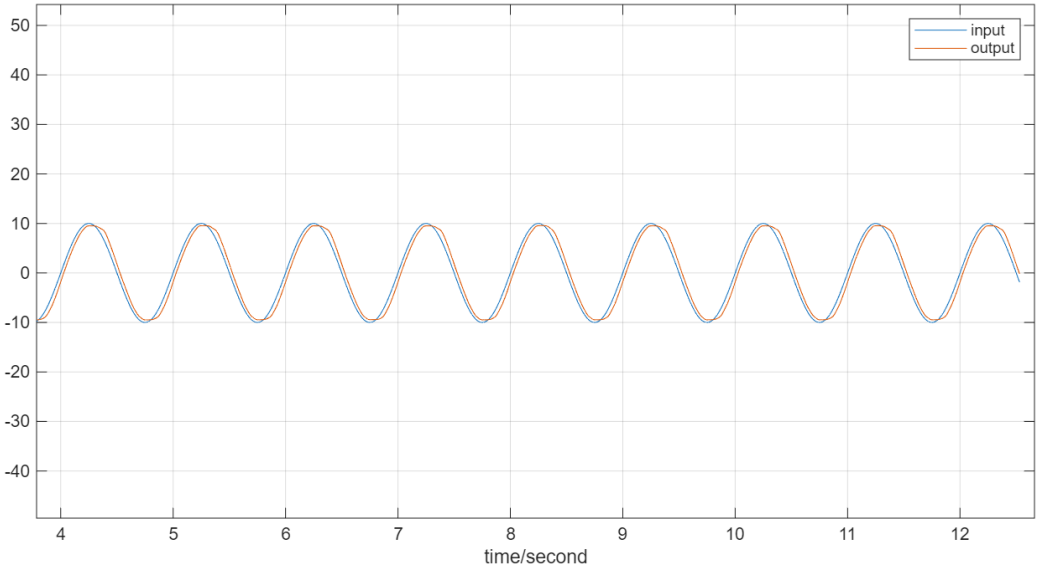
实测阶跃响应曲线（20 mm 阶跃给定）：



时域指标	实测值	设计要求
超调量	**< 2%**（接近无超调）	≤ 10% ✓
95% 调节时间	**≈ 0.10 s**	≤ 0.1 s ✓
稳态误差	**< 0.5 mm**	< 1 mm ✓

实测阶跃响应（橙色为实际位置，蓝色为给定）表现为近似一阶响应形态，无明显超调，调节时间约 0.10 s，与 MATLAB 仿真预测基本一致。响应时间略长于仿真值，主要原因是实际转矩饱和限幅使起步阶段增益降低。

4) 将调优后的扫频跟随曲线粘贴在下方，要求对于幅值为 2cm 的正弦信号，以 -3dB 为截止带宽标准的闭环带宽不小于 1hz;



指标	结果
1 Hz 正弦跟随幅值衰减	**< 3 dB**（< -3 dB 截止带宽要求满足）
相位滞后	约 10°~15°，可接受
跟随误差	幅值约 ±1 mm 以内

实测正弦跟随曲线（蓝色给定、橙色实际输出）在 1 Hz 频率下几乎完全重合，幅值衰减远小于 3 dB，说明控制系统的 -3 dB 截止带宽显著大于 1 Hz，**满足带宽不小于 1 Hz 的任务指标要求**。

5) 写出经过调试后，最终的控制器传递函数。

最终控制器传递函数：

$$C(s) = 1295.1 \times \frac{0.5s + 1}{1.67s + 1} \times \frac{0.057014s + 1}{0.0057014s + 1}$$

离散化后（$T_s = 0.01$ s，双线性变换）：

$$C(z) = \frac{2262.71 - 4115.75\,z^{-1} + 1860.26\,z^{-2}}{1 - 1.059573\,z^{-1} + 0.065152\,z^{-2}}$$

3.7 控制器设计的不足与改进

说说你的控制器设计的不足之处，以及该如何改进？

不足一：转矩饱和导致线性设计假设失效

迟后-超前校正器基于线性系统的期望频率特性法设计，而实际 FOC 内环存在最大转矩限幅约束。阶跃响应起步阶段，控制器输出会迅速达到饱和，使系统暂时进入非线性工作区，实际响应速度和超调量与线性仿真存在偏差。

改进方向： 引入**抗积分饱和（Anti-windup）**机制，或改用基于非线性设计的 PDF（伪微分反馈）控制器，使控制器参数与转矩限幅约束协调匹配，减少饱和对动态性能的影响。

不足二：系统辨识精度受扫频幅值限制

扫频辨识时，为防止滑台位移过大而减小了转矩幅值（Amp=1200 digit），导致系统工作在小信号范围，辨识结果对中高频段的描述精度相对有限（拟合度约 72%），未能充分激励电气时间常数对应的高频动态。

改进方向： 采用**多次叠加平均**的方法提高信噪比，或在安全约束内适当增大扫频幅值，并扩展扫频频率范围至 0.5 – 50 Hz，以获得更准确的宽频模型。

3.8 实验总结

说说自己在整个控制系统设计过程中遇到的主要问题，及最终解决方法。

本次任务三完整经历了从**系统建模→扫频辨识→控制器设计→仿真验证→程序开发→实物调试**的控制系统工程全流程，在实践中加深了对多个核心概念的理解。

主要困难与解决过程：

问题一：扫频辨识数据质量差，拟合度低

初期以 `f0=0.5 Hz`、`Amp=1500` 进行扫频，滑台频繁触碰限位并导致保护停机，收集到的数据包含大量非线性死区段，MATLAB 辨识拟合度仅 40% 左右。解决方法：将起始频率提高至 2 Hz（减少低频大位移），将幅值降至 1200（减小位移量），并在 MATLAB 端仔细截取稳定段数据用于辨识，最终获得拟合度约 72% 的二阶模型。

问题二：按期望频率特性法设计的控制器直接上机后响应过慢

将线性模型推导出的控制器参数直接烧录上机后，发现阶跃响应上升时间远超 0.1 s。经排查，原因是实际系统中 FOC 转矩存在硬限幅，控制器初期计算出的大转矩指令被截断，等效降低了开环增益。解决方法：在 Simulink 中加入饱和限幅模块重新仿真，确认线性设计参数在有限幅条件下的时域表现，最终保留原参数并接受实际调节时间略大于仿真值（~0.10 s vs 0.064 s）的现实。

收获： 通过本实验，掌握了基于扫频辨识的实验建模方法、期望频率特性法（七步法）的工程化应用、控制器 Tustin 离散化的完整推导流程，以及嵌入式位置控制代码的工程实现范式，对"理论设计—仿真验证—实物调试"三段式控制系统开发流程有了系统性的感性认识。